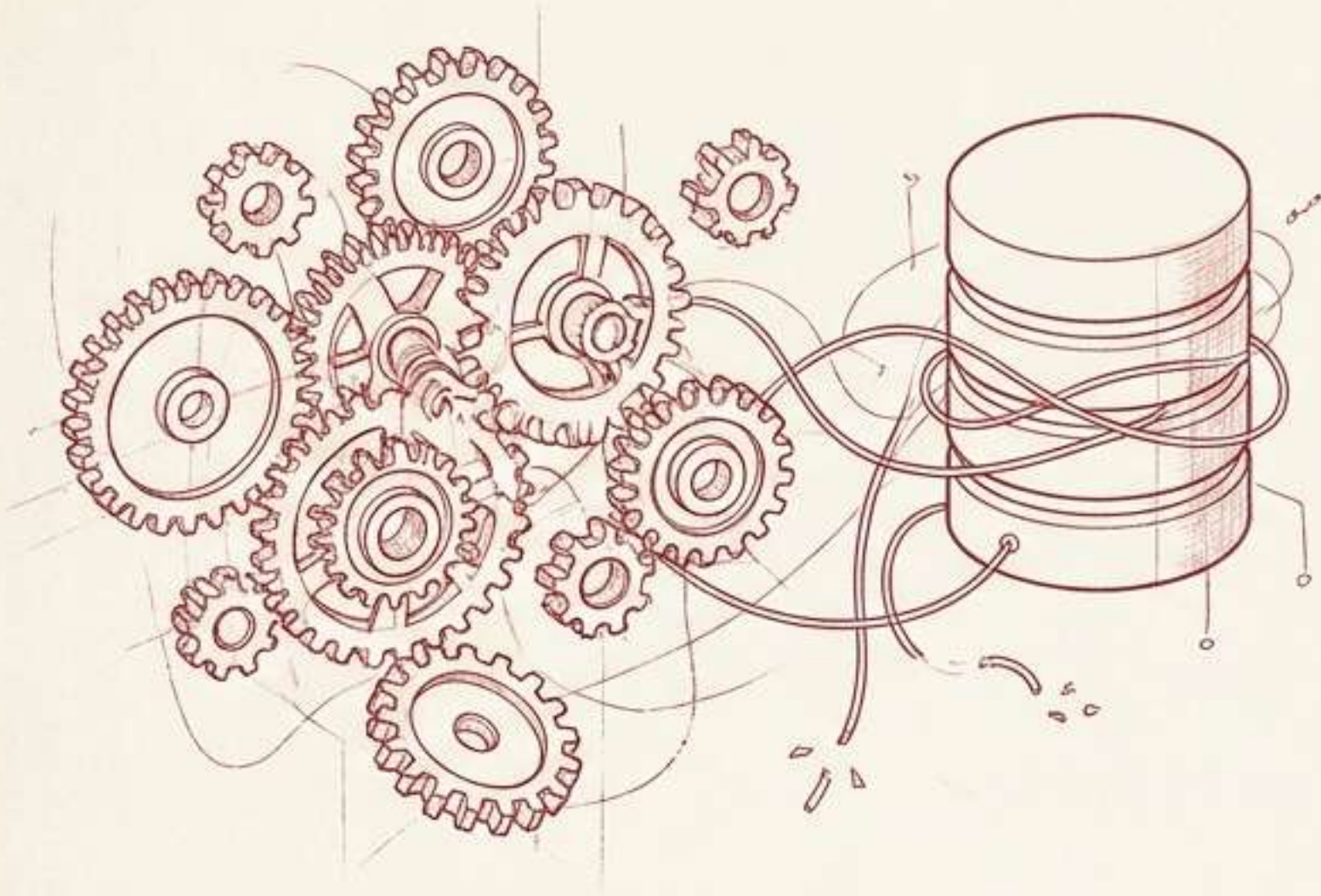




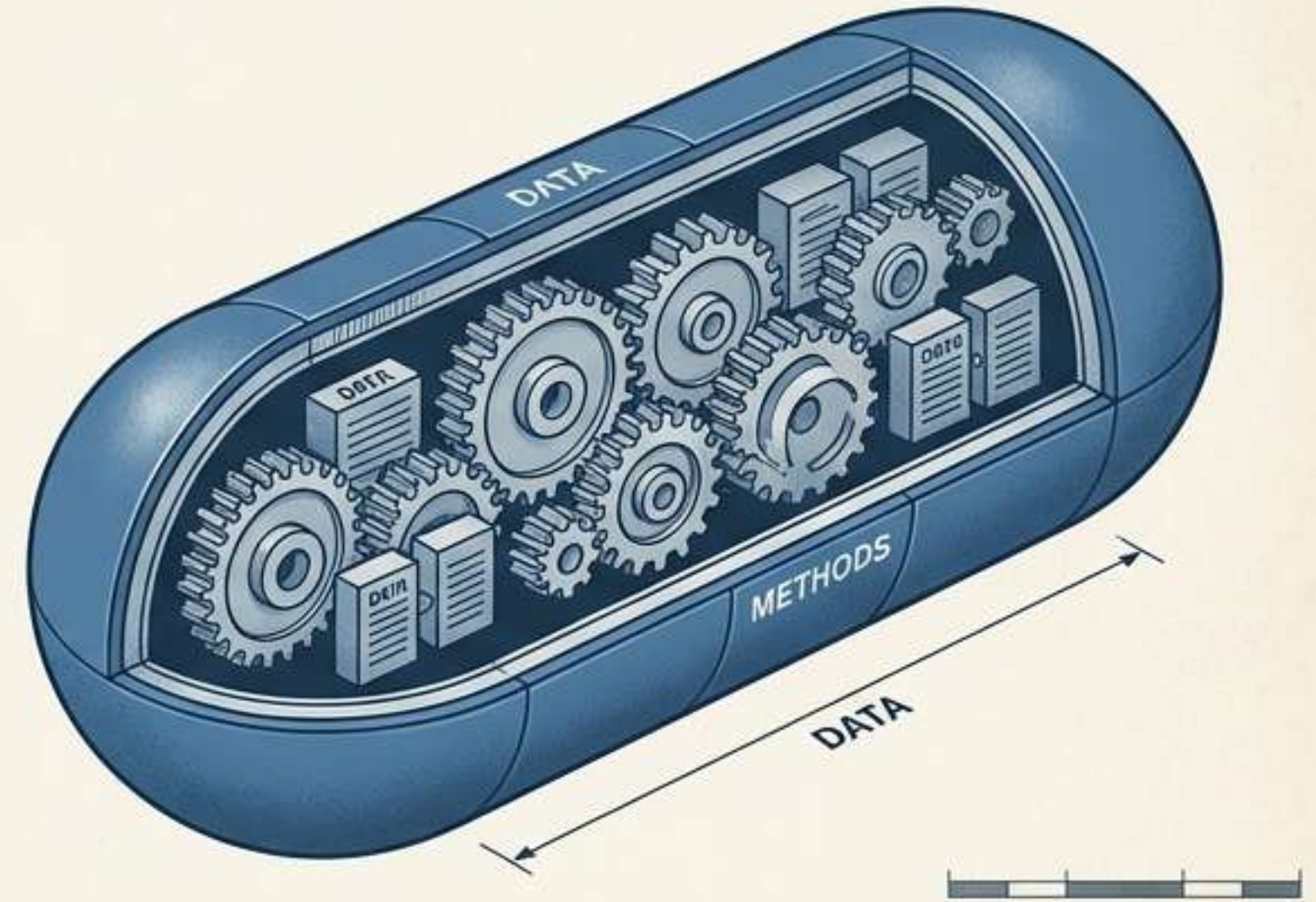
Introducción a la Programación Orientada a Objetos

Dominando la Programación Orientada a Objetos en C#

Años 80: El caos estructurado



POO: El orden encapsulado



Cuatro pilares de la transición:



- **1. Flexible:**
Reutilización sin romper el sistema.



- **2. Natural:**
Modela conceptos del mundo real.

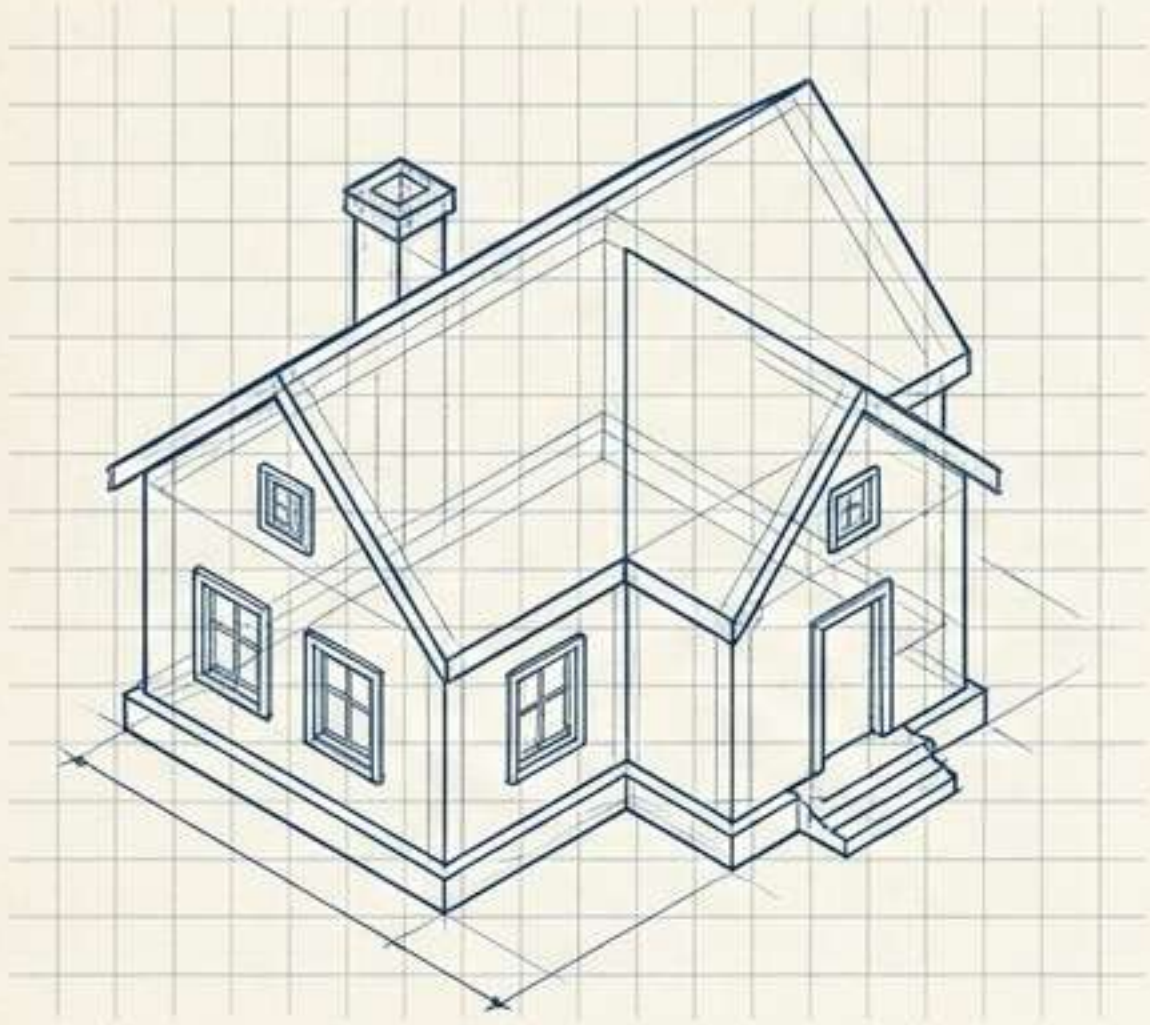


- **3. Segura:**
Oculta datos internos (Encapsulación).



- **4. Testable:**
Cada cápsula tiene una responsabilidad única.

CONCEPTOS FUNDAMENTALES: CLASE vs. OBJETO



La Clase (El Plano)

Define las propiedades y métodos. Es el molde teórico.

```
dotnet new class -n Persona
```

El Objeto (La Realidad)

Instancias reales e independientes nacidas del molde.

```
Persona p1 = new Persona();
```


[El vecindario. Evita que los nombres colisionen con otras librerías.]

[Visible desde cualquier parte del programa.]

```
{ namespace Mi_Proyecto;
```

```
{ public class Persona }
```

```
{
```

```
    public string Nombre { get; set; }}
```

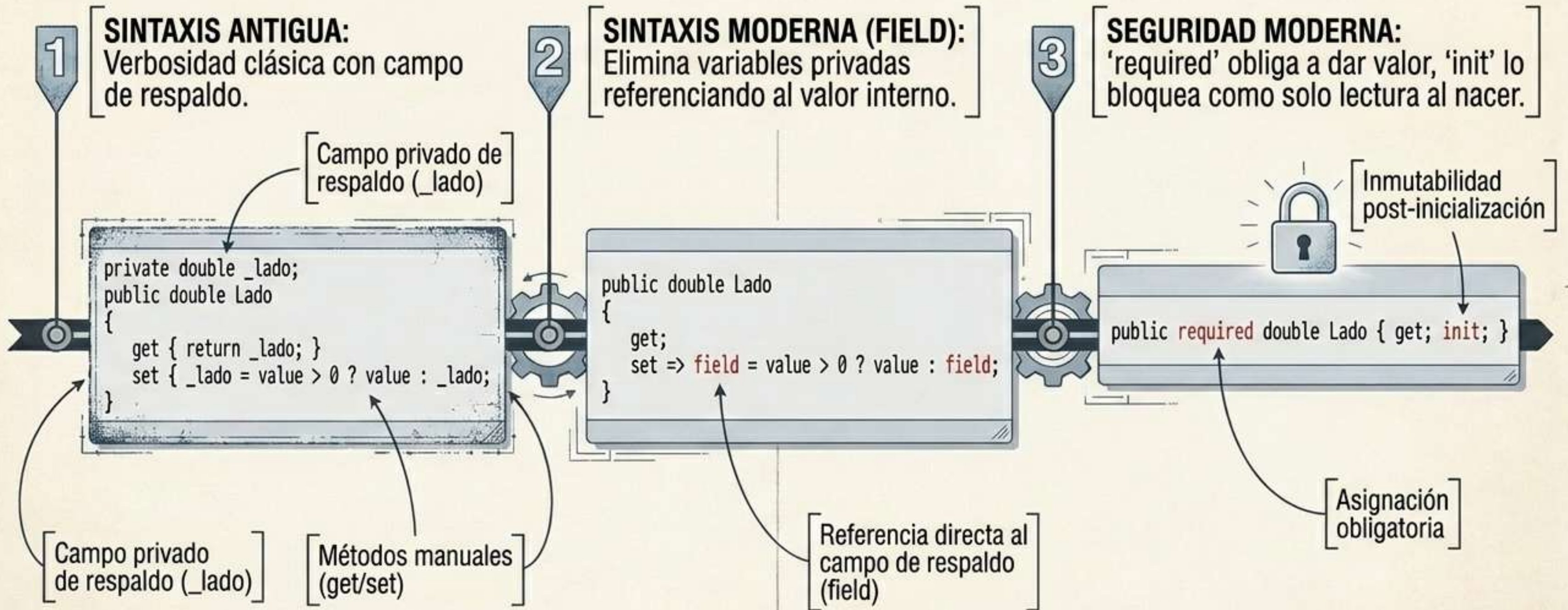
```
    // Métodos...
```

```
}
```

[El nombre del molde (siempre en singular y PascalCase).]

[Propiedad implementada automáticamente. Define el estado del objeto.]

EVOLUCIÓN DE LA SINTAXIS DE PROPIEDADES



EVOLUCIÓN DE CONSTRUCTORES



CONSTRUCTOR CLÁSICO

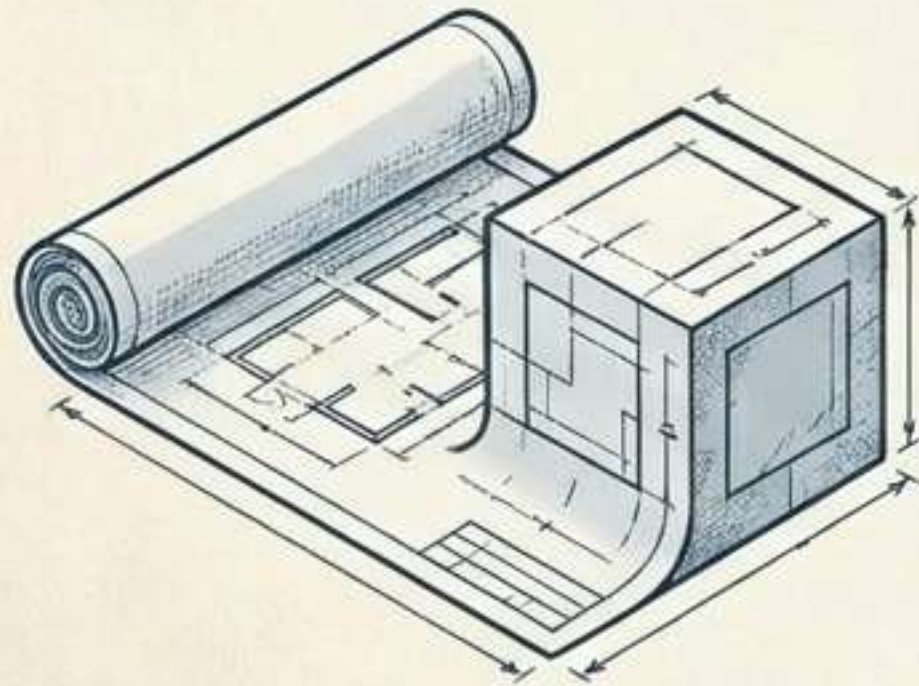
```
public Cuadrado(double lado) { Lado = lado; }
```

Función especial que se ejecuta una sola vez al usar 'new'.



SOBRECARGA

Puedes tener múltiples constructores si cambias el número o tipo de parámetros. El compilador elige el correcto.



CONSTRUCTOR PRIMARIO - C# MODERNO

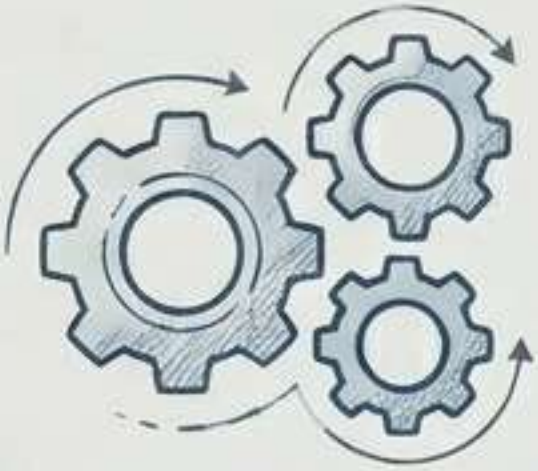
```
public class Cuadrado(double lado)
{
    public double Lado { get; } = lado;
}
```


INICIALIZACIÓN FLEXIBLE: ¿PARÉNTESIS O LLAVES?



INICIALIZACIÓN CON CONSTRUCTOR

```
var c = new Cuadrado(23);
```

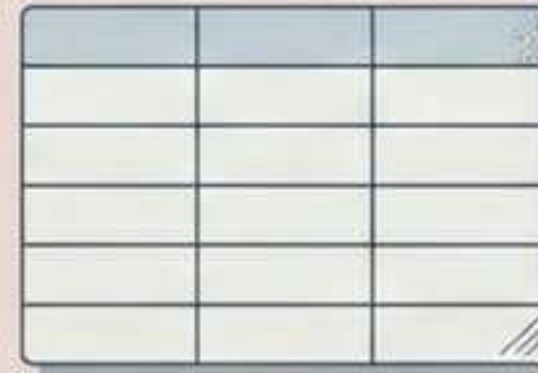


Ideal cuando la inicialización requiere lógica extra, validaciones complejas o inyección de dependencias.



SINTAXIS DE LLAVES

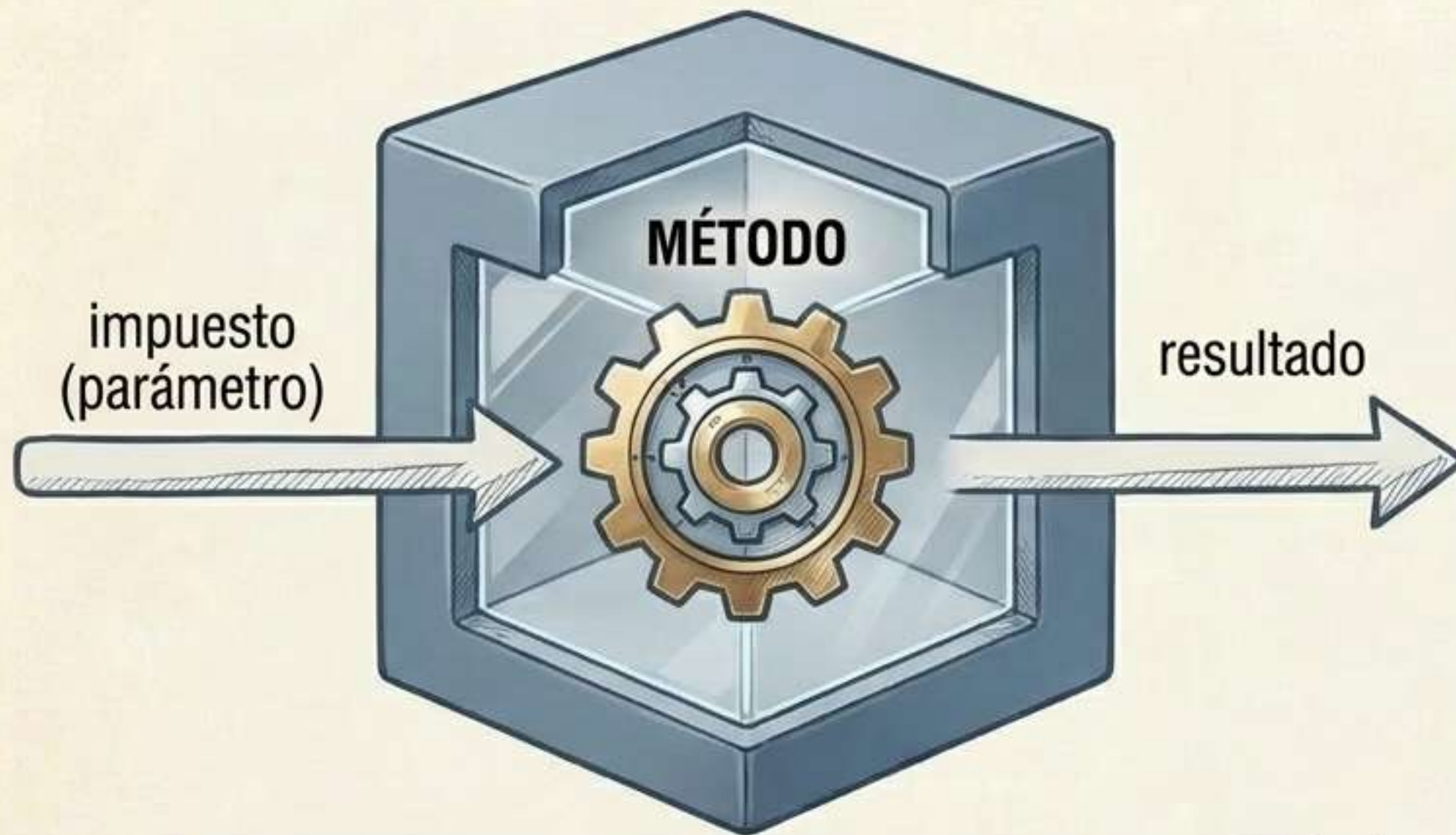
```
var c = new Cuadrado { Lado = 23 };
```



Perfecto para objetos que solo almacenan datos. No requiere definir un constructor previo.

Mezcla inteligente: Usa el constructor para valores obligatorios y las llaves para configurar propiedades opcionales.

MÉTODOS: COMPORTAMIENTO DEL OBJETO



```
public double GetPrecioImpuesto(double impuesto)
{
    return Precio * (1 + impuesto);
}
```

1. **Comportamiento:** Añaden acciones a los objetos.
2. **Estado actual:** Los métodos operan sobre los valores específicos del objeto instanciado.
3. **Sobrecarga:** Mismo nombre, diferentes parámetros (ej. **GetPrecioImpuesto()** vs **GetPrecioImpuesto(0.21)**).

ESCUDOS DE ENCAPSULACIÓN

	 Misma Clase	 Clase Derivada	 Clase Externa	 Otro Proyecto
public	✓	✓	✓	✓
internal (Default)	✓	✓	✓	✗
protected	✓	✓	✗	✗
private	✓	✗	✗	✗

Protege tu estado. Usa 'private' por defecto y solo expón lo que sea estrictamente necesario con 'public'.

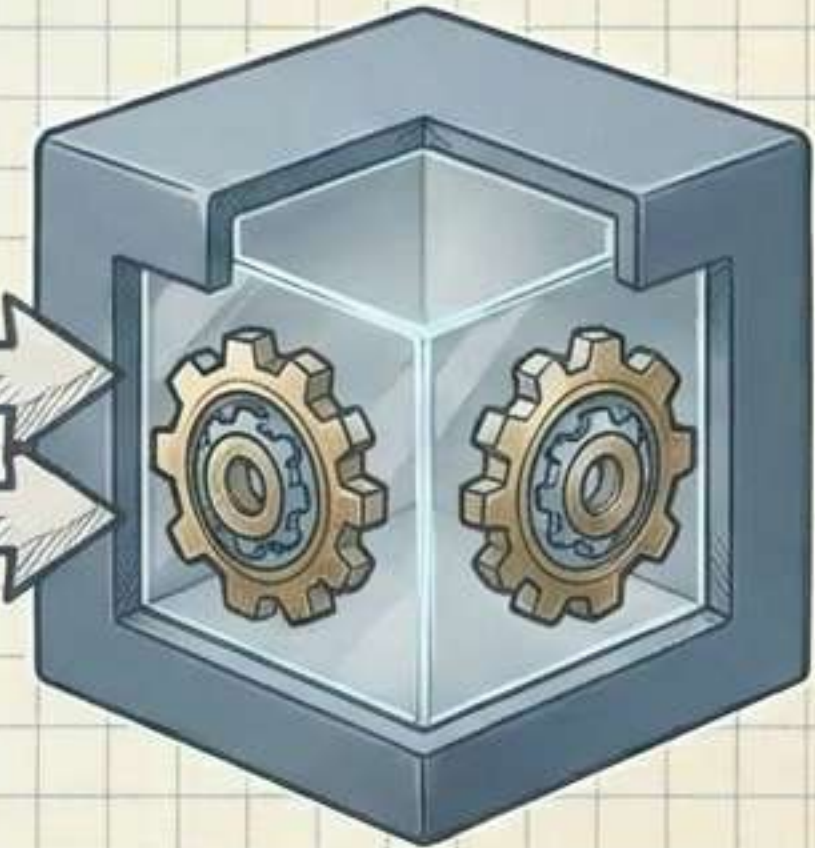
REFERENCIA Y COPIA EN MEMORIA: ASIGNACIÓN Y PARÁMETROS

```
Producto otroProd = prod;
```

prod

otroProd

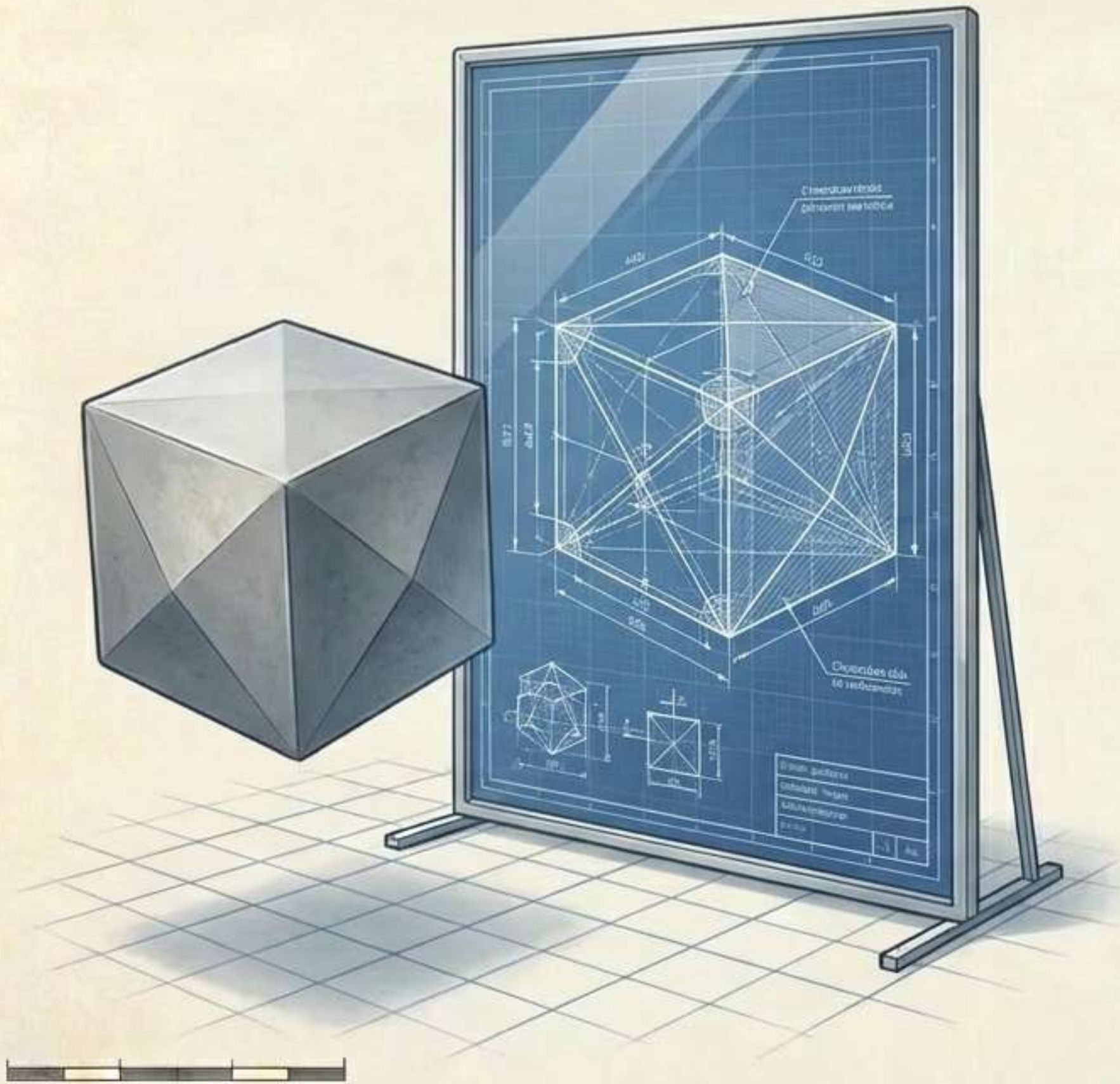
Memoria RAM



Las variables no contienen el objeto, contienen la dirección del objeto. Asignar o pasar por parámetro no duplica el objeto, solo crea una nueva flecha apuntando al mismo lugar.



¡Peligro! Si cambias otroProd.Precio, prod.Precio también cambia.



La Introspección

'**this**' es la referencia que el objeto usa para mirarse a sí mismo desde adentro.

Resolución de Ambigüedad

```
public Cuadrado(double lado)
{
    this.lado = lado;
    // this.lado = propiedad interna
    // lado = parámetro recibido
}
```

En C#, 'this' es implícito, salvo cuando hay ambigüedad de nombres.

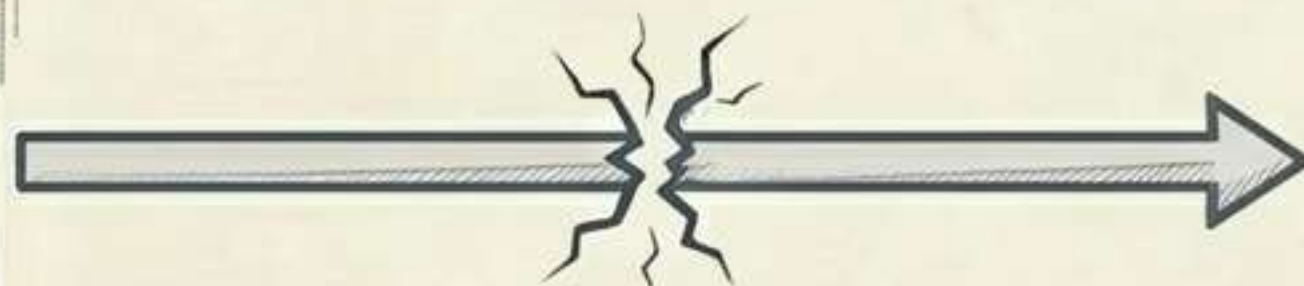
La regla global: static

Lo estático pertenece a la clase, no a los objetos.
(Ej. `Producto.TotalProductos`).

No se puede usar 'this'.



variable
(obj)



El peligro del vacío: **null**

Una referencia que no apunta a ningún objeto.



¡Peligro!

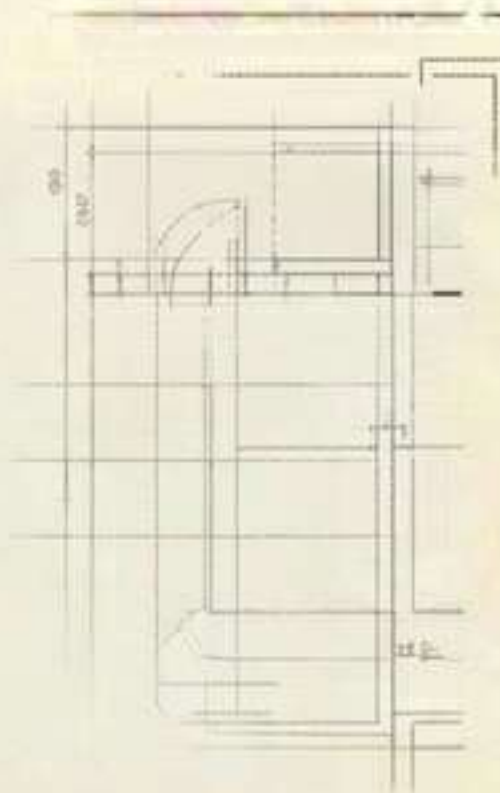
Acceder a una propiedad aquí
lanza **NullPointerException**.
Siempre comprueba **if (obj == null)**.



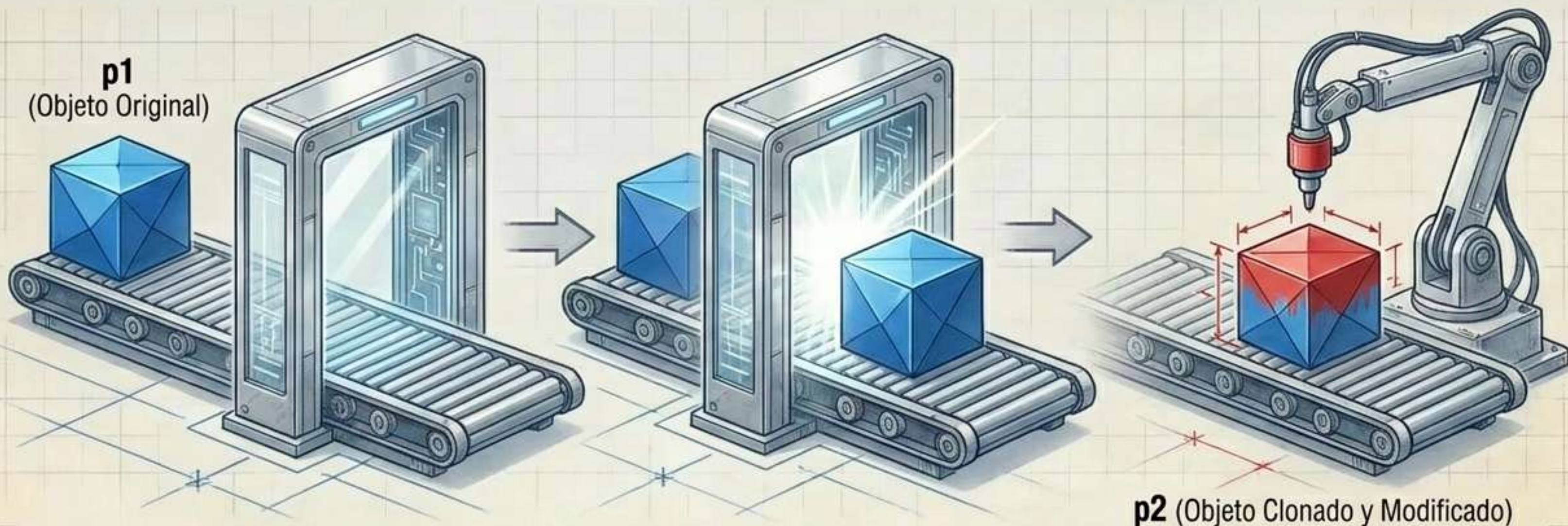
Evolución Inmutable: El tipo “record”

1. **Inmutabilidad total:** Propiedades de solo lectura por defecto. Datos que nacen y no cambian.
2. **Igualdad por valor:** Si dos records tienen los mismos datos, son "==". (Las clases compararían referencias).
3. **Auto-formato:** Incluyen un ToString() preconfigurado que imprime propiedades elegantemente.

```
record Persona(string Nombre, int Edad);
```



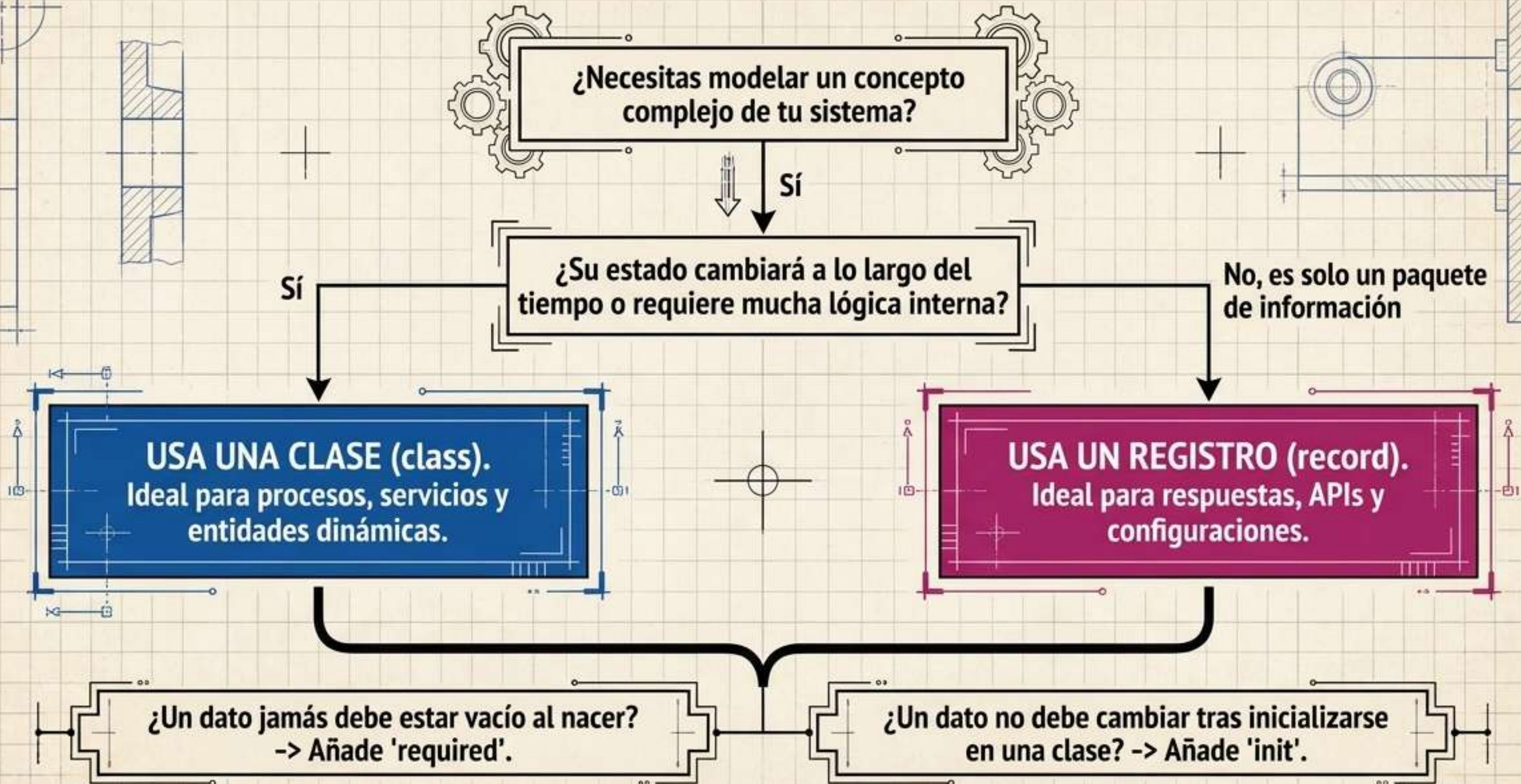
Mutación no acción no destructiva: La palabra clave **'with'**



```
var p2 = p1 with { DNI = "12345678G" };
```

Los records son inmutables. El operador 'with' clona el objeto completo y te permite alterar propiedades específicas al vuelo, devolviendo una instancia completamente nueva y segura.

El árbol de decisiones arquitectónicas en C#



DISEÑA CON INTENCIÓN. DEL PLANO A LA REALIDAD.